QualiAdept

Session 3: Static Testing

Chapter 1: What is Static Testing and why do we look for defects before writing code?

> 🧠 So far we have discussed running tests, executing steps, and verifying applications in operation. This is **Dynamic Testing**. However, a senior QA knows that the most dangerous and expensive bugs can be eliminated without even a single line of code being executed. This is the world **Static Testing**.

1. Definition: Static Testing vs. Dynamic Testing

- **Static Testing:** Evaluation of an artifact (requirements document, design schematic, source code) **without executing** software. It is based on manual analysis (reviews) or automatic code analyzers.
- **Dynamic Testing:** Verifying the software while it is running (on a test environment, with test data entered).

2. What can we test statically? (Artifacts)

In the static phase, the "test object" is not the functional application, but its documentation and representations:

- **User Stories and Business Requirements (SRS):** Detecting inconsistencies, ambiguities and missing data.
- **Mockups / Design Prototypes (Figma/Adobe XD):** Checking visual flows and interface elements.
- **Architecture and System Diagrams:** Checking how the modules are planned to communicate.
- **Source Code (Code Review):** Checking compliance with programming standards and internal logic before compilation.

3. Why is Static Testing a “Gold Mine”? (Economic Benefits)

As we saw in the defect cost curve, the earlier we find a bug, the cheaper it is to fix it:

- **Maximum efficiency:** Finding a bug in a User Story just means changing a text in Jira. It means 0 wasted developer hours and 0 wasted QA hours in dynamic testing.
- **Prevention of production defects:** Over 50% of bugs in production originate from poorly understood or incomplete requirements.
- **Clarity for the entire team:** When QA analyzes static requirements, it forces the Product Owner and developers to align the vision for the product.

💡 QualiAdept Analogy: “Architect’s Blueprint”

> 💡 Imagine you want to build a house.
>
> - **Static Testing:** You check the house plan drawn on paper by an architect. You notice that the architect forgot to put a door in the bathroom or that he drew a staircase that stops in a wall. You erase the line with an eraser and redraw it correctly. Cost: **5 seconds and 0 RON**.
> - **Dynamic Testing:** You build the entire house, lay the bricks, paint it, and only when you move in (during the construction phase) do you try to get into the bathroom and realize there is no door. You have to break down the wall, rebuild the structure, and paint it again. Cost: **Thousands of euros and weeks of work**.

🕵️ Thinking Exercise: "The Static Eye"

> 📝 Statically analyze the following requirement written by a Product Owner:
> *"The system will process the user's payment quickly and send a confirmation email if everything is OK."*
>
> - What is vague/ambiguous in this requirement?
> - What error case is completely missing?
> - How would you rephrase this requirement to be development-ready?

Chapter 2: Review Techniques and Types (From

Discussions to Formal Inspections)

> 🎯 After I understood *What* is static testing and *Why* is vital (saving the budget), we must learn *How* is done. In the world of QA, reviewing a document is not just about reading it diagonally. According to the ISTQB standard, there are four main types of reviews, classified from the most relaxed (informal) to the most strict (inspection).

1. The 4 Types of Reviews (ISTQB Standard)

The choice of review type depends on the maturity of the process in the company, the time available, and the criticality of the software.

A. Review-ul Informal (Informal Review)

- **How it works:** There's no documented process. You just ask a colleague, "Hey, can you take a look at this Test Case and see if it makes sense?"
- **Who drives it:** Nobody in particular. It's usually done in the format of *Pair Programming* or *Pair Testing*.
- **Scope:**Finding obvious defects quickly and cheaply, without wasting time with bureaucracy.

B. Walkthrough

- **How it works:** The author of the document organizes a meeting and presents the document (or code) to the others, step by step.
- **Who drives it:** The author (e.g. the Product Owner presents the new business requirement to the development and QA team).
- **Scope:** Creating a common understanding, getting feedback and educating the team about the new module. It is a useful review to align the team (knowledge sharing).

C. Review-ul Tehnic (Technical Review)

- **How it works:** A documented process, supported by technical experts (without management, so there is no pressure). Checklists are used.
- **Who drives it:** A trained Moderator or technical expert (NOT the author).
- **Scope:** Ensures that the document or code complies with the company's technical standards (e.g.: Is the code architecture correct? Has the security standard been met?).

D. Inspection

- **How it works:** The most formal, rigid, and documented type of review. There are strict entry and exit criteria. Members read the document before the meeting and come with the defects already noted. Metrics are collected (e.g., how many defects were found on the page).
- **Who drives it:** A **Moderator** dedicated and trained. The author is not allowed to lead the inspection.
- **Scope:** Finding critical flaws in documents of utmost importance (e.g. financial algorithms, medical systems).

2. Roles in a Formal Inspection

To pass the ISTQB exam (and to organize an effective QA meeting), you need to know who sits at the table of an inspection:

- **Author:** The person who wrote the document/code. Their role is to answer questions and fix any defects found.
- **The moderator:** The person leading the meeting ensures that discussions do not deviate (time management) and mediates conflicts.
- **Reviewer:** This is where it comes in. **The Tester (QA)** Your role is to come prepared and identify flaws based on your experience.
- **Scribe (The Scribe/Secretary):** Write down all the flaws and decisions made during the meeting.
- **The manager:** Usually, *NOT* participate in the inspection, so that the perpetrator does not feel evaluated or threatened.

🔘QualiAdept Analogy: “Releasing a Book”

> 🧠 Imagine you are writing a novel (Code/Application):
>
> - **Review Informal:** You give the first chapter to your spouse to read quickly and tell you if they like the action.
> - **Walkthrough:** You gather your friends in the living room and you read the book out loud to them. As you read, they say, "Wait, didn't this character die in chapter 2?" You brainstorm together.
> - **Technical Review:** Send your manuscript to another professional writer. He or she doesn't judge the story, but checks that you've followed the grammar rules and storytelling

techniques.

- **Formal Inspection:** You send the book to a Publishing House. There, a Chief Editor (Moderator) gathers a Proofreader (Tester) and a Critic. They apply a strict checklist. You just sit and listen to what needs to be fixed before printing.

💡 Did you know that...?

💡 In successful IT teams there is a concept called **„Egoless Programming”**? This says that the flaws found in a review belong to *PRODUCT*, not to the *AUTHOR.* When you do a static review, you never say, “You’re wrong here,” but rather, “There’s an inconsistency in this paragraph.” The point of the review is to attack the problem, not the person!

🕵️ Thinking Exercise: "Choose the Right Tool"

📝 Imagine you are a QA Lead at a company. What type of review (Informal, Walkthrough, Technical or Inspection) would you organize for the following situations?

- A junior developer just wrote 10 lines of code to change the color of a button and wants you to make sure he chose the right shade.
- The Product Owner has finished writing the general requirements document for a new application and wants the entire team to understand it.
- The team designed the card payment processing logic for an online store with millions of users (the company's most critical module).

Chapter 3: Recap, Conclusions and Case Study (QA Review in Action)

🎯 We have reached the end of Session 3. If so far we have learned the theory behind static

> testing and the processes through which we carry it out (reviews), now it is time to put the theory into practice. A successful QA does not just read standards, but actively intervenes to "clean" the project documentation.

1. Comprehensive Review: What did we learn in Session 3?

- **Preventive mindset:** I understand that testing is starting *before* of writing code. Static testing is about prevention, while dynamic testing is about cure.
- **Quality Economy (Shift Left):** Finding a defect in a document costs 10-100 times less than finding it in production, because it does not involve hours of programming, compiling, and re-testing.
- **Spectrum of Reviews:** We learned to adapt the rigor of the verification to the importance of the document:
  - *Informal:* Fast, collegial, cheap.
  - *Walkthrough:* The author explains, the team learns and comments.
  - *Technical Review:* Experts check technical standards.
  - *Formal Inspection:* Rigid process, led by a Moderator, with strict metrics (for critical systems).
- **Egoless Programming:** I have assimilated the principle that we criticize the code/document, never the person who wrote it.

2. Case Study: How do we do a QA Review in real life?

Let's simulate a Grooming / Walkthrough session in an Agile team. The Product Owner (PO) brings a new functionality for an online store to the team.

A. Initial Requirement (As written by the PO)

- **Title (User Story):** Apply discount code.
- **Description:** "As a user, I want to be able to enter a promotional code in the shopping cart to pay less."
- **Acceptance Criteria:**
  - The user enters the code "REDUCERE20".
  - The system deducts 20% of the total order.
  - The user can complete the payment.

B. Tester Intervention (Static QA Review)

If the programmer takes this requirement and writes the code directly, the application will be full of vulnerabilities. As a QualiAdept tester, you raise your hand and ask the following questions (identifying static defects):

- **Defect de Incompletitudine (Edge Cases):**
  - *QA asks:* "What happens if the user enters the code twice? The discount becomes 40%?"
  - *QA asks:* "What happens if the user enters 'discount20' (in lowercase)? Is it Case Sensitive?"
- **Business Logic Flaw:**
  - *QA asks:* "Does the 20% discount only apply to products or also to shipping? (If it also applies to shipping, the company loses money)."
- **Defect regarding the Lack of Negative Scenarios:**
  - *QA asks:* "What error message do we display if the code has expired or was entered incorrectly?"

C. Refined Requirement (After QA Review)

Thanks to your intervention, the Product Owner modifies the document. The new form looks like this:

- **Revised Acceptance Criteria:**
  - The 20% discount applies **only for the subtotal of products**, excluding shipping fee.
  - Code **it is not** Case Sensitive (also accepts "discount20" and "DISCOUNT20").
  - The system allows the application of a **single code** per order. If a second code is entered, the first is overwritten and the user is notified.
  - If the code is invalid/expired, the message appears in red text: "The code entered is not valid."

Case Study Conclusion: With 5 minutes of discussion and static testing, you saved days of work. You prevented bugs that would have allowed customers to get products for free (by bundling codes) and ensured a clear user experience. This is the added value of a Senior QA!

3. QualiAdept Conclusions: “The Laws of Static Testing”

- **A vague requirement is a pending bug.** Never let an ambiguity slip into the development phase.

- **The review attacks the problem, not the person.** Use a constructive tone. The goal is product quality, not demonstrating superiority.
- **Look beyond Happy Path.** Document authors usually describe how things work when everything is perfect. Your job is to ask:*"What if...?"*

🚀 What's next?

> 💡 With this session we conclude **Module 1: Fundamentals and Tester Mindset.**
> Now you have the thinking, vocabulary (ISTQB) and strategy needed.
> In **Module 2**, we are moving to enterprise-level tools! We will enter **Jira** (the most used management software in the world) and we will learn how to write our Test Cases and report bugs using dedicated extensions such as **Zephyr** or **TestRail.** Get ready for a lot of platform action!

Bonus Chapter: Automated Static Testing (When Robots Do Code Review)

> 🧠 So far, we have looked at static testing from the human perspective: meetings, discussions on documents, formal inspections. This is half the story. In modern IT companies, Static Testing is also done in **automated ways**, using software tools that "read" the code written by programmers without ever running it.

> As a QA Manual, you may not configure these tools yourself, but you need to know how to read their reports to know how "risky" the code you are about to test is.

1. What are Static Code Analyzers?

There are special programs (e.g.: **SonarQube**, **ESLint**, **Checkstyle**) that scan the application's source code as soon as the programmer hits the "Save" button or tries to send the code to the server. They look for bad patterns, broken rules, or known vulnerabilities.

2. What types of defects do these “robots” find?

Automated static analysis is absolutely brilliant at finding hidden technical issues that the human eye would easily miss:

- **Security Vulnerabilities:** I find hardcoded passwords or unsecured connections to databases.
- **Dead Code:** Identifies variables or functions that have been written, but are never used in the program (taking up memory for nothing).
- **Violation of Coding Standards:** Check if the programmer followed the company's rules (e.g. if they put parentheses where they should or if they named the variables correctly).
- **Cyclomatic Complexity:** It is a metric that counts how many decisions (if/else) there are in a single function. If a function has too many branches, the static analyzer warns:*"Warning, this piece of code is too complex and will be a nightmare to test and maintain!"*.

3. The Role of QA in Automated Static Analysis (Quality Gate)

In an advanced project, the QA (or Test Manager) collaborates with the DevOps team to set a **Quality Gate**.

This is a strict rule: if SonarQube (the static analyzer) says that the new code has a security score below B, or has more than 5 technical bugs, **the code is automatically rejected.** It never reaches the test environment, and you, as a tester, don't waste your time on poorly written code.

💡 QualiAdept Analogy: “The Spellchecker”

Think about when you are writing a document in Microsoft Word.

- **Manual Static Testing** It means printing the document and giving it to a colleague to read and tell you if your ideas make sense (Walkthrough/Inspection).
- **Automated Static Testing** is the wavy red line that appears **instantly** under a word when you ate a letter (ex:*„progrmare”*). Word didn't "run" your book, it didn't understand its story, but it scanned the text using a predefined dictionary and found a syntax error in milliseconds.

Conclusion: Robots (static analyzers) are perfect for finding syntax errors and broken technical rules (the red line in Word), but only a human QA (Manual Static Testing) can read a requirements document to tell you that the application "story" doesn't make sense!

🚀 To Module 2...

Now that you have a "black belt" in prevention and static testing, you are truly ready to enter the **Module 2**, where we will learn how to document our work and report bugs (even those found statically) using **Jira**.